GRUPO KND: Gabriel Erick, Camila Carniel & Bruno Germanetti

PARTE 02: Documento de Especificação de Requisitos: Projeto SIP

1. Introdução

O **SIP** é uma plataforma social e de registro pessoal voltada para entusiastas de vinhos. O sistema permite o catálogo de experiências sensoriais, acompanhamento de métricas de consumo e interação com uma rede de contatos para recomendações de rótulos, funcionando como um "diário de bordo" digital para cada taça.

---

2. Requisitos Funcionais (RF)

Título do Requisito	Descrição	Prioridade	Critérios de Aceitação
RF01: Registro de Experiência (Check-in)	O sistema deve permitir que o usuário registre o consumo de uma bebida, anexando uma foto da garrafa/taça, nome do rótulo, tipo de de bebida e suas particularidades, local e data.	**Crítica**	1. O registro deve exigir obrigatoriamente uma foto e uma nota (1-5). 2. Deve permitir a inserção de um comentário descritivo (notas de degustação).
RF02: Feed de Atividade Social	O sistema deve as postagens e avaliações dos amigos que o usuário segue dentro do **SIP**.	**Alta**	1. As postagens devem ser exibidas em ordem cronológica inversa. 2. Deve exibir o nome do usuário, foto do vinho e a nota atribuída.
RF03: Painel de Estatísticas Pessoais	O sistema deve consolidar o total de bebidas degustados e calcular a nota média global do usuário.	**Alta**	1. Os dados devem ser atualizados imediatamente após cada novo registro. 2. O usuário deve conseguir visualizar o histórico completo na aba "Experiências".
RF04: Recomendações de Amigos	O sistema deve destacar rótulos bem avaliados por contatos próximos do usuário na tela inicial do **SIP**.	**Baixa**	1. Devem aparecer vinhos com nota acima de 4.0 estrelas.

Título do Requisito	Descrição	Prioridade	Critérios de Aceitação
			2. O card deve indicar explicitamente qual amigo recomendou o vinho.
RF05: Catálogo/Exploração de Rótulos	O sistema deve fornecer uma ferramenta de busca para pesquisar bebidas na base de dados global do **SIP**.	**Baixa**	1. A busca deve funcionar por nome do vinho ou região. 2. Se o vinho não existir, deve permitir a criação de um novo registro manual.

---

3. Requisitos Não Funcionais (RNF)

Título do Requisito	Descrição	Prioridade	Critérios de Aceitação	Métricas Mensuráveis
RNF01: Desempenho de Carregamento	O tempo de resposta para carregar o feed de fotos no **SIP** deve ser otimizado para não prejudicar a experiência.	**Alta**	As imagens devem ser carregadas de forma assíncrona (lazy loading).	Tempo de carregamento da página inicial < 2.5 segundos em rede 4G.
RNF02: Design e Acessibilidade	A interface do **SIP** deve seguir o padrão *Dark Mode* para uso confortável em ambientes de baixa luminosidade.	**Média**	O contraste entre texto e fundo deve seguir os padrões WCAG.	Taxa de contraste mínima de 4.5:1 para textos normais.
RNF03: Escalabilidade de Mídia	O armazenamento deve suportar o crescimento exponencial de fotos de alta resolução enviadas pelos usuários.	**Alta**	Compressão automática de imagens no upload sem perda visível de qualidade.	Redução de 60% no peso original do arquivo sem pixelização.
RNF04: Disponibilidade do Serviço	O **SIP** deve estar operacional para registros imediatos, especialmente em horários de pico.	**Crítica**	Minimização de janelas de manutenção em noites e finais de semana.	SLA de disponibilidade de 99.0%.

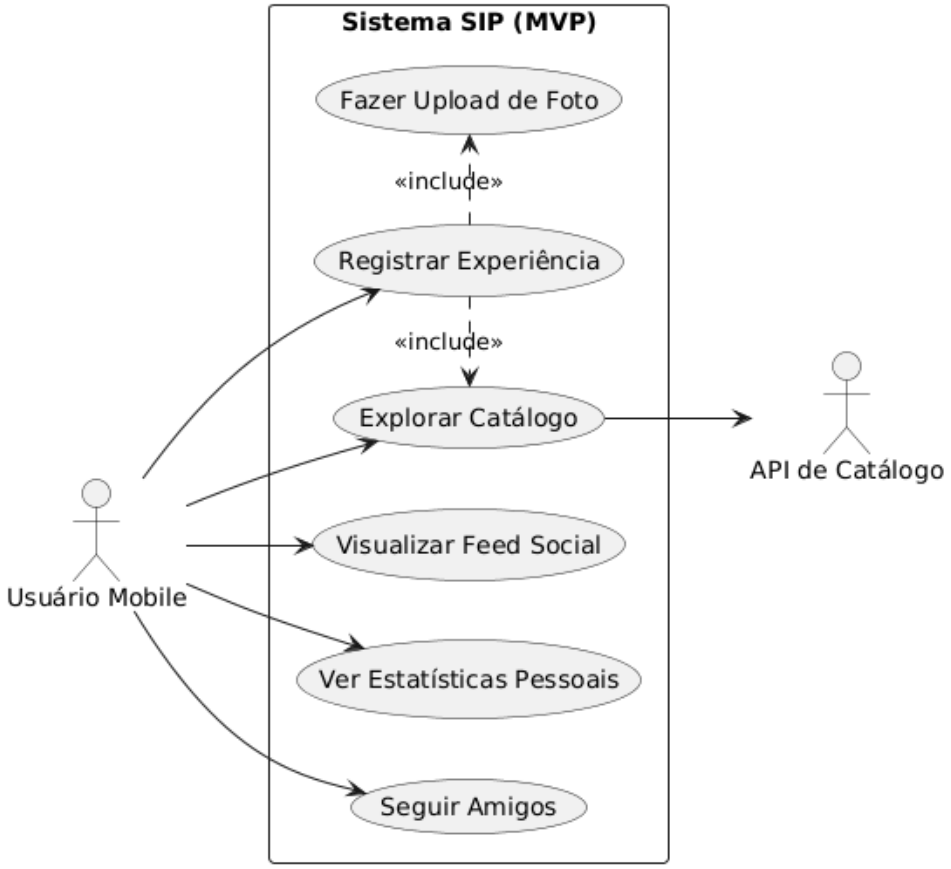
4. Modelagem de User Stories (Visão Ágil)

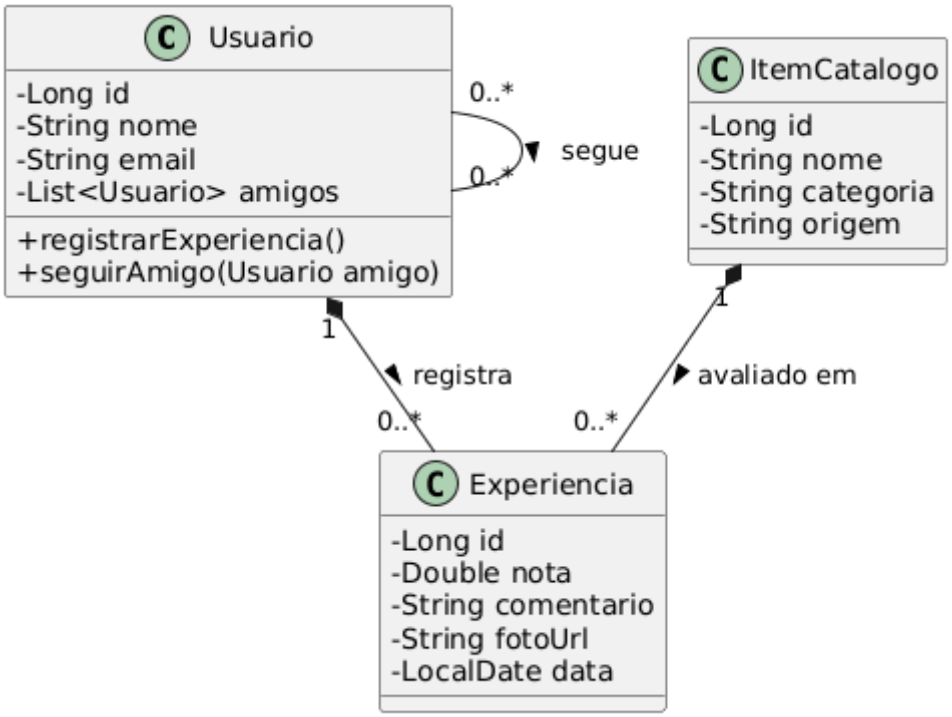
- **US01 - Registro Rápido e Visual:** Como um entusiasta de boas experiências, quero usar o SIP para registrar a minha bebida atual rapidamente com uma foto e uma nota, para que eu possa focar no momento social sem esquecer o que consumi.

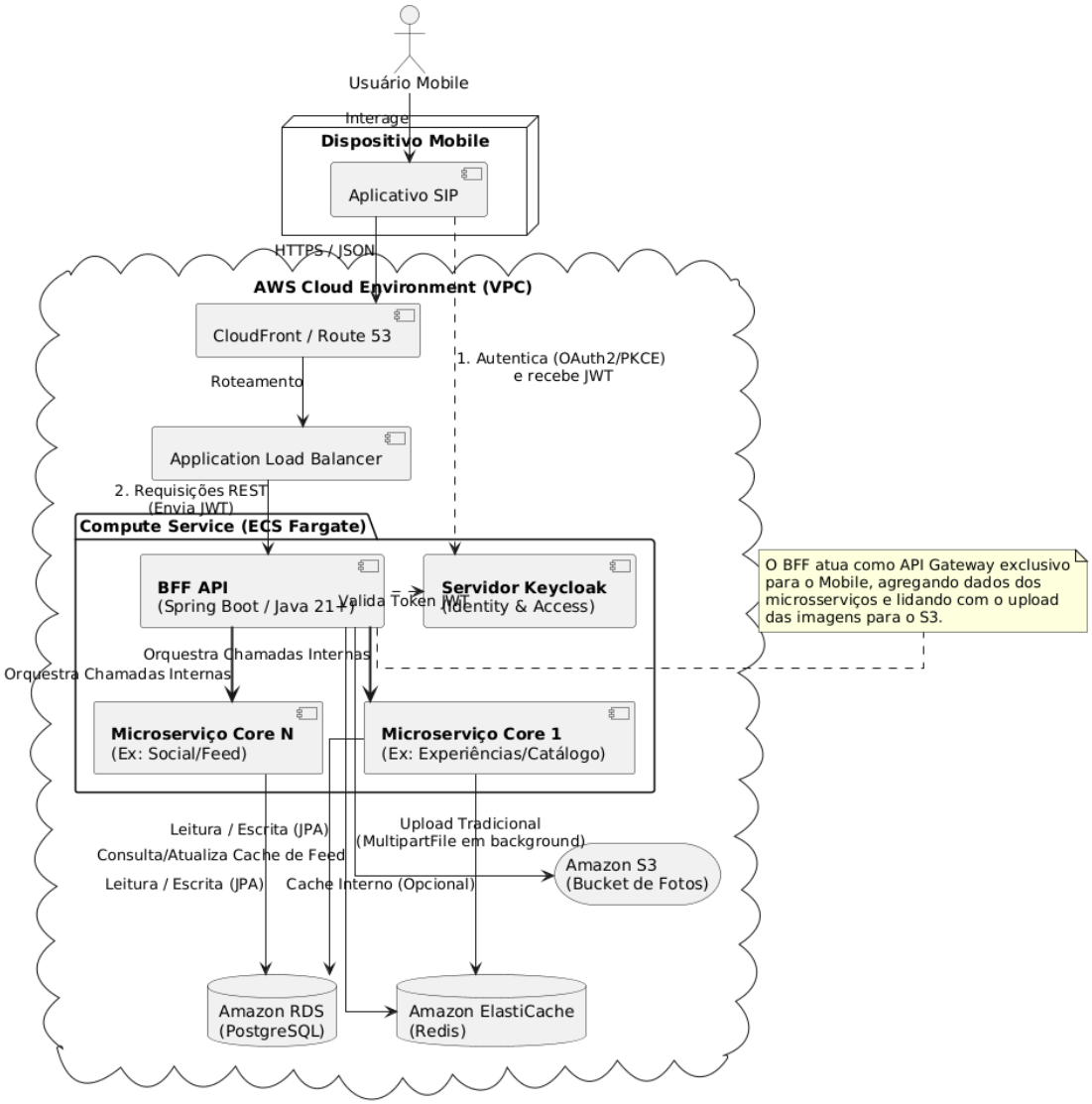
- **US02 - Descoberta pelo Feed Social:** Como um usuário explorando novas opções (como em um bar ou restaurante), quero abrir o SIP e ver o feed com as avaliações recentes dos meus amigos, para descobrir novos rótulos e ter confiança na minha próxima escolha.

- **US03 - Memória Afetiva e Histórico:** Como um colecionador de momentos, quero revisitar minha linha do tempo (timeline) e minhas estatísticas pessoais no SIP, para relembrar as ocasiões especiais, com quem eu estava e quais itens mais me agradaram.

05. Diagramas




Usuario
-Long id
-String nome
-String email
-List<Usuario> amigos
+registrarExperiencia()
+seguirAmigo(Usuario amigo)
0..*
0..*
segue
ItemCatalogo
-Long id
-String nome
-String categoria
-String origem
1
registra
0..*
1
avaliado em
0..*
Experiencia
-Long id
-Double nota
-String comentario
-String fotoUrl
-LocalDate data



4. Justificativa da Arquitetura Adotada

A arquitetura do **SIP** foi projetada para equilibrar a **agilidade de desenvolvimento inicial (Time-to-Market)** com a **capacidade de escala horizontal**, tratando o vinho não apenas como um dado, mas como uma experiência visual e social.

4.1. Critérios de Escolha e Alinhamento com Requisitos

A decisão por uma arquitetura baseada em **Containers (Docker/Fargate)** e **Serviços Gerenciados (AWS)** baseia-se nos seguintes pilares:

- **Alta Disponibilidade (RNF04):** O uso do **AWS ECS Fargate** permite que o backend em Java opere em múltiplas zonas de disponibilidade. Caso um servidor falhe, a infraestrutura se recupera automaticamente, garantindo que o usuário consiga registrar seu vinho no exato momento da degustação.
- **Performance e Escalabilidade de Mídia (RNF01 e RNF03):** Como o SIP é um app intensivo em fotos, a separação entre o servidor de aplicação (Java) e o armazenamento de arquivos (**Amazon S3**) é vital. Isso evita que o upload de imagens pesadas degrade o desempenho das consultas ao banco de dados.

- **Segurança e Conformidade (LGPD):** Delegar a autenticação ao Keycloak reduz o risco de vazamento de dados sensíveis, uma vez que o sistema não armazena senhas localmente, utilizando tokens JWT criptografados para validar cada interação.

4.2. Por que Java 21+ com Spring Boot?

A escolha do stack tecnológico foi fundamentada na **maturidade e tipagem forte**:

1. **Ecossistema Robusto:** O Spring Boot oferece integração nativa com quase todos os serviços AWS, acelerando a implementação de filas e conexões com bancos de dados.
2. **Manutenibilidade:** A tipagem estática do Java e o padrão de Inversão de Controle (IoC) facilitam a expansão do sistema por múltiplos desenvolvedores, garantindo que novas regras de negócio (como sistemas de gamificação ou badges) possam ser adicionadas sem quebrar funcionalidades existentes.

4.3. Trade-offs e Decisões de Design

Ao projetar a arquitetura, avaliámos o dilema Monólito vs. Microsserviços:

- **A Escolha:** Optámos por uma arquitetura orientada a Microsserviços aliada ao padrão BFF (*Backend For Frontend*).

- **Justificativa:** Embora a abordagem de microsserviços introduza uma ligeira complexidade inicial de configuração e orquestração de rede, a adoção de um BFF desenvolvido em Spring Boot (Java 21+) centraliza e simplifica drasticamente a comunicação com a aplicação móvel. O dispositivo móvel não precisa de conhecer a topologia interna do sistema; comunica exclusivamente com o BFF (via chamadas REST) e com o Keycloak (para autenticação JWT).

4.4. Persistência Poliglota: O uso de RDS e Redis

A arquitetura descarta o uso exclusivo de um único banco de dados.

- **PostgreSQL (RDS)** foi escolhido pela integridade referencial — essencial para garantir que uma "Experiência" esteja sempre vinculada a um "Vinho" e um "Usuário" válidos.
- **Redis (ElastiCache)** foi introduzido para mitigar o custo computacional de buscas repetitivas. Sem o cache, cada vez que um usuário abrisse a aba "Explorar", o banco sofreria uma carga alta de leitura, prejudicando a performance global.

Conclusão da Justificativa

A arquitetura adotada minimiza o esforço de gerenciamento de infraestrutura (*Ops*) ao utilizar serviços *serverless* e gerenciados, permitindo que o foco da equipe permaneça na evolução das funcionalidades do **SIP**. É uma estrutura que nasce preparada para suportar desde 100 até 100.000 usuários sem necessidade de reescrita do código core.

5. Matriz de Rastreabilidade de Requisitos (MRR)

Esta matriz garante que cada funcionalidade mapeada no início tenha um correspondente técnico na modelagem e na infraestrutura AWS.

ID Requisito	Título	Item de Modelagem (DCL)	Componente Arquitetural
RF01	Registro de Experiência	Classe Experiencia	AWS S3 (Fotos) / RDS (Dados)
RF02	Feed Social	Classe Usuario (Self-relation)	AWS RDS / Redis (Cache de Feed)
RF03	Estatísticas Pessoais	Métodos em Usuario	Spring Boot (Business Logic)
RF05	Explorar Rótulos	Classe ItemCatalogo	Amazon ElastiCache (Busca)
RNF01	Performance	-	CloudFront / Redis
RNF04	Disponibilidade	-	Multi-AZ (ECS Fargate)

6. Glossário do Projeto SIP

O alinhamento de termos evita ambiguidades entre os desenvolvedores Java e os Product Managers.

- **Check-in (ou Registro):** O ato de fotografar, avaliar e salvar uma degustação no sistema.
- **Rótulo (Label):** Referência técnica a uma bebida específica na base de dados, no caso do vinho seria: nome + vinícola + safra + Safe.
- **Safe (Vintage):** Utilizada para bebidas do tipo vinho, refere-se ao ano em que as uvas foram colhidas; campo crítico para a unicidade do vinho.
- **JWT (JSON Web Token):** O "passaporte" digital gerado pelo Keycloak que identifica o usuário nas requisições ao backend.
- **Fargate:** Modelo operacional da AWS para rodar containers sem gerenciar servidores, usado para hospedar o Spring Boot, Redis, Keycloak e etc.
- **Entidade (Entity):** Classe Java que representa uma tabela no banco de dados (ex: Entidade bebida).

---

7. Análise de Riscos Iniciais (Arquitetura)

Identificamos os pontos de falha potenciais e as estratégias de mitigação para o **SIP**.

Risco	Impacto	Mitigação
Latência no upload de fotos	Alta (Experiência do Usuário)	Implementar o envio da imagem em background no aplicativo mobile e configurar o back-end (Spring Boot) para receber arquivos via MultipartFile e processar o envio ao Amazon S3 de forma assíncrona, evitando o bloqueio da interface do usuário.
Custo inesperado de Nuvem	Média (Financeiro)	Configurar limites de escalabilidade no AWS Fargate e usar políticas de ciclo de vida no S3 para comprimir fotos antigas.
Inconsistência na Base de Vinhos	Média (Qualidade dos Dados)	Implementar um algoritmo de busca fonética ou similaridade (ex: Levenshtein) para evitar duplicidade de rótulos.
Dependência do Keycloak	Baixa (Vulnerability)	Utilizar padrões de mercado (OpenID Connect) para facilitar a migração para outro provedor de identidade, se necessário.